🛒 Retail Sales Data Analysis Project – End-to-End Guide

---

1. Project Objective

Goals:

- Analyze sales data to identify business trends.
- Understand customer purchasing behavior and regional sales performance.
- Detect top-performing products, categories, and customers.
- Build predictive models to forecast sales.
- Provide actionable insights for business growth and optimization.

---

2. Load Dataset & Initial Inspection

import pandas as pd

```
df = pd.read_csv('/content/sales_data.csv')
df.info()
df.describe()
df.head()
```

Insights:

- Dataset contains 500 records and 11 columns.
- Includes categorical (Region, Product, Category), numerical (Price, Quantity, TotalSales), and temporal (OrderDate, year) features.

---

3. Data Cleaning

a. Check Missing & Duplicate Values

```
print(df.isnull().sum())
print("Duplicates:", df.duplicated().sum())
```

b. Handle Missing / Duplicates

```
df.drop_duplicates(inplace=True)

df.fillna({'Region': 'Unknown', 'Category': 'Misc'}, inplace=True)
```

c. Convert Datatypes

```
df['OrderDate'] = pd.to_datetime(df['OrderDate'])
```

d. Create New Features

```
df['Month'] = df['OrderDate'].dt.month_name()

df['DayOfWeek'] = df['OrderDate'].dt.day_name()

df['Revenue_per_Item'] = df['TotalSales'] / df['Quantity']
```

---

4. Exploratory Data Analysis (EDA)

```
import matplotlib.pyplot as plt

import seaborn as sns


plt.figure(figsize=(8,5))

sns.countplot(x='Region', data=df)

plt.title('Sales Count by Region')

plt.show()
```

Univariate Analysis

```
sns.histplot(df['TotalSales'], kde=True)

plt.title('Distribution of Total Sales')

plt.show()
```

Bivariate Analysis

```
sns.scatterplot(x='Price', y='Quantity', data=df, hue='Category')

plt.title('Price vs Quantity by Category')

plt.show()
```

Multivariate Analysis

```
corr = df[['Quantity', 'Price', 'TotalSales']].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm')
```

---

5. Outlier Detection

IQR Method

```
Q1 = df['TotalSales'].quantile(0.25)
Q3 = df['TotalSales'].quantile(0.75)
IQR = Q3 - Q1
df = df[(df['TotalSales'] >= Q1 - 1.5 * IQR) & (df['TotalSales'] <= Q3 + 1.5 * IQR)]
```

---

6. Feature Engineering

```
df['Sales_per_Region'] = df.groupby('Region')['TotalSales'].transform('sum')
df['Customer_Spend'] = df.groupby('CustomerID')['TotalSales'].transform('sum')
df['Profit_Estimate'] = df['TotalSales'] * 0.25  # assuming 25% profit margin
```

---

7. Descriptive Statistics

```
print("Mean Sales:", df['TotalSales'].mean())
print("Median Sales:", df['TotalSales'].median())
print("Standard Deviation:", df['TotalSales'].std())
```

---

8. Statistical Analysis

a. Correlation Test

```
corr = df[['Quantity', 'Price', 'TotalSales']].corr(method='pearson')
print(corr)
```

b. Hypothesis Example:

H0: Price does not affect Total Sales.
H1: Price has a significant effect on Total Sales.

```
from scipy.stats import linregress

slope, intercept, r_value, p_value, std_err = linregress(df['Price'], df['TotalSales'])

print(f"P-value: {p_value}, R²: {r_value**2}")
```

---

9. Time-Series Analysis

Monthly Sales Trend

```
monthly_sales = df.groupby(df['OrderDate'].dt.to_period('M'))['TotalSales'].sum()

monthly_sales.plot(marker='o', figsize=(10,5), title='Monthly Sales Trend')
```

---

10. Customer Analysis

Top Customers

```
top_customers =
df.groupby('CustomerID')['TotalSales'].sum().sort_values(ascending=False).head(10)

sns.barplot(x=top_customers.values, y=top_customers.index, palette='mako')

plt.title("Top 10 Customers by Revenue")

plt.show()
```

---

11. Product & Category Insights

```
product_sales =
df.groupby('Product')['TotalSales'].sum().sort_values(ascending=False).head(10)

sns.barplot(x=product_sales.values, y=product_sales.index)

plt.title('Top Selling Products')

plt.show()


category_sales = df.groupby('Category')['TotalSales'].sum()
```

```
category_sales.plot(kind='pie', autopct='%1.1f%%', title='Sales Share by Category')
```

---

12. Predictive Modeling (Regression – Forecast Sales)

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score

X = df[['Quantity', 'Price', 'year']]
y = df['TotalSales']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = RandomForestRegressor(n_estimators=200, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

---

13. Classification Model (High vs Low Sales Orders)

```
df['Sales_Label'] = df['TotalSales'].apply(lambda x: 'High' if x >
df['TotalSales'].median() else 'Low')

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

X = df[['Quantity', 'Price', 'year']]
y = df['Sales_Label']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
clf = RandomForestClassifier(n_estimators=200, random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

---

14. Feature Importance

```
importances = model.feature_importances_
plt.barh(X.columns, importances)
plt.title('Feature Importance for Total Sales Prediction')
plt.show()
```